

---

## Data Structures

---

BS (CS) \_Fall\_2025

### Lab\_05 Manual



### Learning Objectives:

1. Singly Linked List

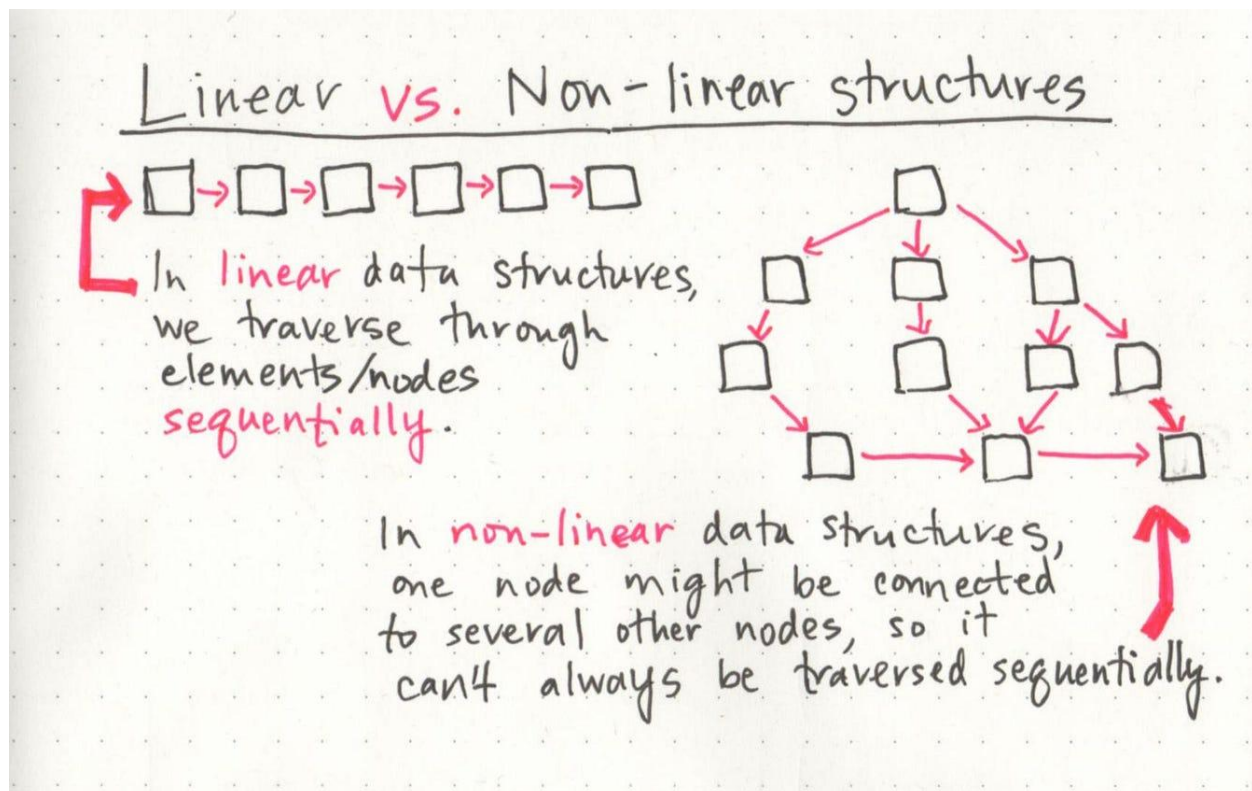
# Singly Linked List

## Objective:

- To understand the concept of linked lists.
- To implement a singly linked list in C++.
- To perform operations such as insertion, deletion, traversal, and searching.

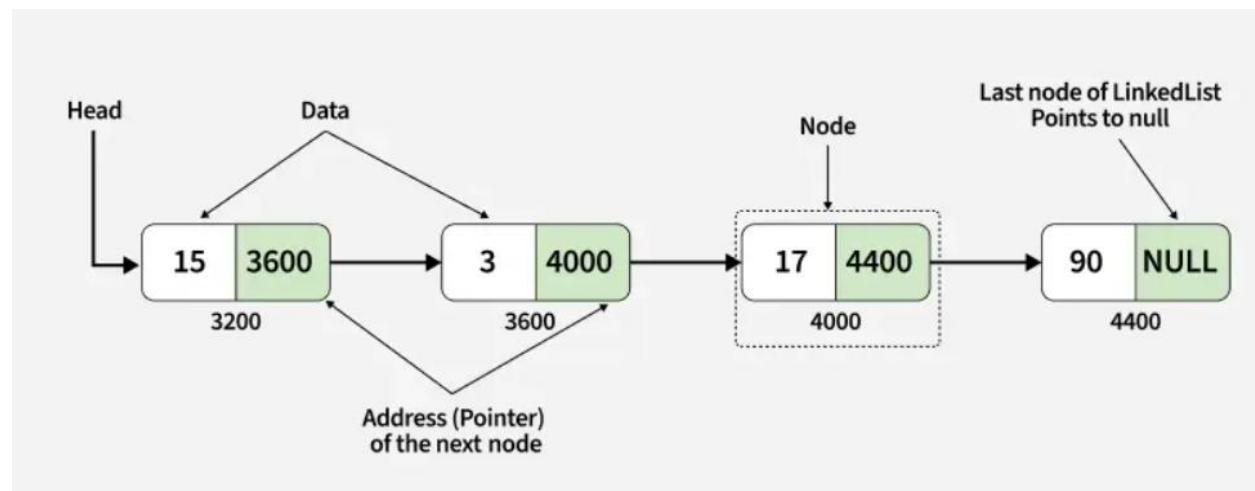
## What is a Linked List?

linked lists are linear data structures, meaning there is a sequence and an order to how they are constructed and traversed. In order to get to the end of the list, we have to go through all of the items in the list in order, or sequentially. Linear structures, however, are the opposite of non-linear structures. In non-linear data structures, items don't have to be arranged in order, which means that we could traverse the data structure non-sequentially.



## Parts of a Linked List:

A **linked list** is a linear data structure where elements (called *nodes*) are connected using pointers. Unlike arrays, linked lists do not require contiguous memory locations. The starting point of the list is a reference to the first node, which is referred to as the head. Nearly all linked lists must have a head, because this is effectively the only entry point to the list and all of its elements, and without it, you wouldn't know where to start! The end of the list isn't a node, but rather a node that points to null, or an empty value.



Each node in a singly linked list consists of:

- **Data field** : stores the actual data.
- **Pointer (next)** : stores the address of the next node.

### Advantages:

- Dynamic memory allocation (no need to predefine size).
- Efficient insertion and deletion (especially in the middle).

### Disadvantages:

- Extra memory for pointers.
- Sequential access only (no direct indexing).

## A Simple Linked List Class:

- We use two classes: Node and List
- Declare Node class for the nodes
  - **data:** double-type data in this example
  - **next:** a pointer to the next node in the list

```
// Node structure
struct Node {
    int data;          // data field
    Node* next;       // pointer to next node
};
```

- Declare List, which contains
  - **head:** a pointer to the first node in the list.Since the list is empty initially, head is set to NULL

```
// Linked List class
class LinkedList {
private:
    Node* head;

public:
    LinkedList();           // Constructor
    ~LinkedList();         // Destructor

    void insertAtHead(int val);
    void insertAtTail(int val);
    void deleteNode(int val);
    bool search(int val);
    void display();
};
```